

Special Participation B: Cursor on HW12

Jincheng Ou

December 7

1 Prompt

Input Prompt:

You are a deep learning expert. Based on the questions shown in the uploaded image, complete the coding task. Follow the instructions in `q_vae.ipynb` to complete: implement the `TODO` for the reparameterization trick in `sample_gaussian` in `utils.py`; implement `negative_elbo_bound` in `models/vae.py`. You need to clone the code first (according to the instructions in `q_vae.ipynb`).

2 Generated Answers by Cursor

2.1 Part 1: Reparameterization Trick (`utils.py`)

The following code implements the `sample_gaussian` function using the reparameterization trick:

```
1 def sample_gaussian(m, v):
2     """
3     Element-wise application reparameterization trick to sample from
4     Gaussian
5
6     Args:
7         m: tensor: (batch, ...): Mean
8         v: tensor: (batch, ...): Variance
9
10    Return:
11        z: tensor: (batch, ...): Samples
12    """
13    #####
14    # TODO: Modify/complete the code here
15    # Sample z
16    #####
17    # Reparameterization:  $z = m + \sqrt{v} * \text{eps}$ ,  $\text{eps} \sim N(0, I)$ 
18    eps = torch.randn_like(m)
19    z = m + torch.sqrt(v) * eps
20    return z
```

Outstanding performance. Cursor demonstrated perfect "one-shot" capability, correctly implementing the mathematical constraints that other models struggled with. The code is concise, Pythonic, and fully functional.

Correct Implementation.

2.2 Part 2: Negative ELBO Bound (models/vae.py)

The following code implements the `negative_elbo_bound` function:

```
1 def negative_elbo_bound(self, x):
2     """
3     Computes the Evidence Lower Bound, KL and, Reconstruction costs
4
5     Args:
6         x: tensor: (batch, dim): Observations
7
8     Returns:
9         nelbo: tensor: (): Negative evidence lower bound
10        kl: tensor: (): ELBO KL divergence to prior
11        rec: tensor: (): ELBO Reconstruction term
12    """
13    mean, var = self.enc.encode(x)
14    z = ut.sample_gaussian(mean, var)
15    logits = self.dec.decode(z)
16
17    #####
18    # TODO: Modify/complete the code here
19    # Compute negative Evidence Lower Bound and its KL and Rec
20    decomposition
21    #
22    # Note that nelbo = kl + rec
23    #
24    # Outputs should all be scalar
25    #####
26    # Monte-Carlo estimate of reconstruction term and KL divergence
27    kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1]).
28    mean()
29    rec = (-ut.log_bernoulli_with_logits(x, logits)).mean()
30    nelbo = kl + rec
31    #####
32    # End of code modification
33    #####
34    return nelbo, kl, rec
```

Perfect handling of scalar reduction.
The implementation is highly
efficient, chaining the operations
cleanly.

3 Executive Summary

Cursor demonstrated exceptional performance on this Deep Learning assignment, effectively one-shotting both the reparameterization trick and the ELBO loss function implementation.

Key Observations:

- **One-Shot Capability:** The LLM successfully generated correct, production-ready code for both tasks without requiring follow-up corrections.
- **Mathematical Accuracy:** Cursor correctly interpreted the VAE mathematical formulation. It accurately applied the reparameterization formula and correctly identified that the reconstruction term requires a negative sign (Negative Log-Likelihood).

- **Handling Constraints:** Cursor correctly applied `.mean()` to both the KL divergence and the reconstruction loss. This ensured the outputs were scalars as explicitly requested in the docstring ("Outputs should all be scalar").
- **Code Management:** Cursor integrates the functions of Git and GitHub, allowing users to directly execute Git commands within the IDE (for example, I have made Cursor itself perform operations like `git clone` to obtain a local copy of the code repository). Cursor does an excellent job. At the same time, when executing Git commands, the system will pause to ask for the user's input.
- **Misconceptions/Hallucinations:** No hallucinations or misconceptions were found. The model showed a solid understanding of PyTorch(e.g., `torch.randn_like`) and the broadcasting mechanics required for the VAE.